

S O U M A

Full application report

From product conception to architecture, integrations, data design, assistants, and operational readiness — print edition.

Generated **2026-04-16T23:11:26Z** · Confidential draft

Repository: souma-store (Next.js App Router)

Document class: technical + product narrative

Estimated print length: ~65–85 pages (engine, margins, and source tree size)

Abstract

This report consolidates the Souma Store application as implemented in source: user journeys, route map, API inventory, persistence model, AI features, and risk-aware recommendations. It is suitable as an onboarding artifact, design review appendix, or lightweight compliance pack when paired with your organization's policies.

Table of contents

1. **Executive summary**
2. **Vision, positioning, and conception**
3. **Stakeholders and personas**
4. **Functional scope overview**
5. **Quality attributes (non-functional requirements)**
6. **Solution architecture**

7. 7. Frontend architecture (App Router)
8. 8. State management
9. 9. Authentication and sessions
10. 10. Commerce, pricing modes, and inventory
11. 11. Checkout, logistics fields, and payments
12. 12. AI assistants: behavior and safeguards
13. 13. Analytics: shopping sequences
14. 14. Data architecture and persistence
15. 15. API surface (summary)
16. 16. Security, privacy, and compliance posture
17. 17. Deployment model
18. 18. Testing strategy (recommendations)
19. 19. Roadmap themes
20. 20. Internationalization, locales, and assistant copy decks
21. 21. Repository layout and layering conventions
22. 22. Catalog data model and shop presentation
23. 23. Product detail page: composition and assistants
24. 24. Cart, persistence, and client state lifecycle
25. 25. Checkout UX: wilaya, commune, shipping presentation
26. 26. Payments: Chargily integration principles
27. 27. Wishlist domain model and UX
28. 28. Reviews, ratings, and moderation considerations
29. 29. Site feedback instrument
30. 30. Blog subsystem and editorial workflow
31. 31. Legal, trust, and policy surfaces
32. 32. Performance engineering checklist
33. 33. Accessibility baseline
34. 34. Engineering process: migrations, typing, and CI
35. 35. Glossary and extended definitions
36. 36. Appendices (routes, APIs, use cases, risks, env, schema, source index)

Executive summary

Souma Store is a production-grade e-commerce experience built on **Next.js 15** and **React 19**, combining a rich merchandising front end, authenticated accounts, Algerian-market checkout (wilaya/commune), **Chargily** payments, and an **AI-assisted shopping layer** with multilingual responses and telemetry for continuous improvement.

This document narrates the product from conception through architecture, data design, operational concerns, and a structured catalog of use cases suitable for stakeholders, engineers, and auditors. Diagrams use plain figures so they render reliably in PDF engines.

- **Primary runtime:** Node.js serverless (Vercel-style) with Route Handlers.
- **Persistence:** PostgreSQL via Drizzle ORM.
- **Auth:** Better Auth with Google OAuth and credential flows.
- **State:** Redux Toolkit for cart, wishlist, quick view, product-details UI.
- **Intelligence:** Google Gemini and/or OpenRouter-backed flows with catalog grounding.

Vision, positioning, and conception

Souma targets a modern, mobile-first storefront with editorial polish: large imagery, confident typography, and fast interaction patterns (quick view, sticky assistant, expressive product pages). Conception emphasized three pillars: *trust* (clear pricing modes, policies, reviews), *speed* (client caching patterns, optimistic UI), and *insight* (shopping sequences + assistant telemetry).

The name and visual language evoke warmth and energy—orange accents on deep navy—while keeping commerce flows conventional so new shoppers remain oriented.

Design principle: progressive disclosure. Advanced modes (jomla pricing, assistant) enhance the core journey without blocking basic browse-to-buy.

Stakeholders and personas

Guests discover the catalog, compare prices, and may purchase depending on checkout configuration. **Registered users** retain wishlists, reviews, and personalized surfaces. **Operators** monitor sequences and assistant metrics. **Developers** extend Drizzle models, API routes, and UI modules within the App Router structure.

CHAPTER 4

Functional scope overview

Major domains include catalog browsing, product detail, cart, checkout + payment, wishlist, authentication, blogs and static content, feedback and reviews, admin sequence analytics, and AI assistants (global + product-scoped).

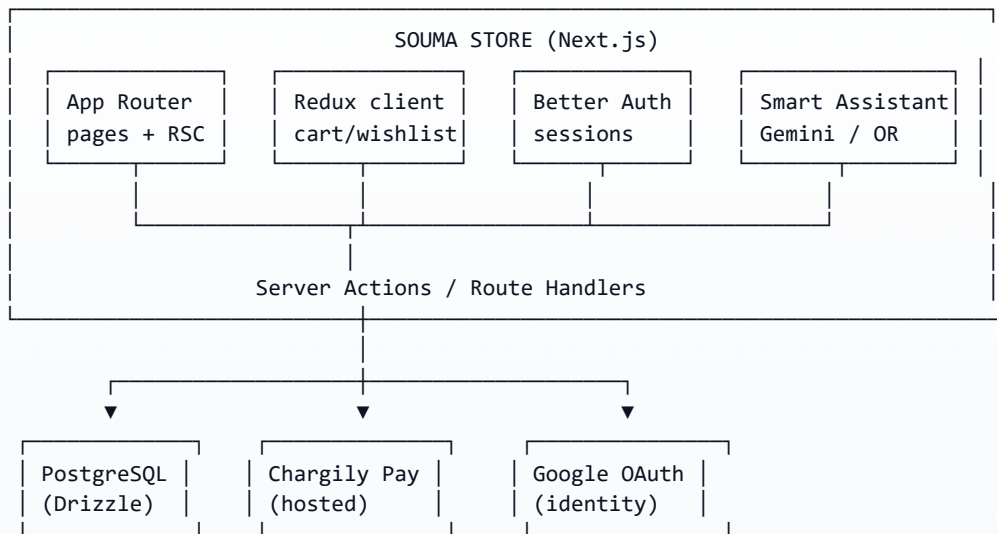
Quality attributes (non-functional requirements)

Attribute	Target behavior	Implementation notes
Performance	Fast TTFB on static segments; responsive interactions	Next.js streaming, image optimization, minimal client JS on content pages
Security	Session integrity; server-side payment creation	Better Auth, server-only secrets, Chargily server API
Maintainability	Typed end-to-end	TypeScript, Drizzle schema as source of truth
Observability	Funnel and assistant metrics	shopping_sequence + assistant_search_telemetry tables
i18n readiness	Assistant speaks ar/fr/en/dz	Locale heuristics + template banks in route handler

Solution architecture

The system follows a modular monolith pattern: a single deployable Next.js application containing UI, API routes, and server libraries. External systems are limited to well-bounded integrations (payments, LLM providers, OAuth).

Figure 1 — System context (C4 Level 1)



Edge vs Node: Middleware performs a narrow cookie presence check for matched paths; sensitive authorization should still occur in server components or route handlers per Better Auth guidance.

Frontend architecture (App Router)

Routes live under `src/app/(site)` with nested layouts supplying session, theme, Redux Provider, cart persistence, modals, and assistant shells. Pages remain mostly server components where feasible; interactive islands mount client components for cart, swiper carousels, and assistants.

- **Layouts:** shared chrome, fonts, analytics hooks (as present).
- **Composition:** components/ tree mirrors domains: Shop, Checkout, Cart, Common.
- **Styling:** Tailwind CSS utility system with design tokens in tailwind config.

CHAPTER 8

State management

Redux Toolkit configures reducers for quick view, cart, wishlist, and product-details slices. Selectors are typed via `useAppSelector`. Cart persistence bridges storage and the store to avoid hydration mismatch and flicker.

Authentication and sessions

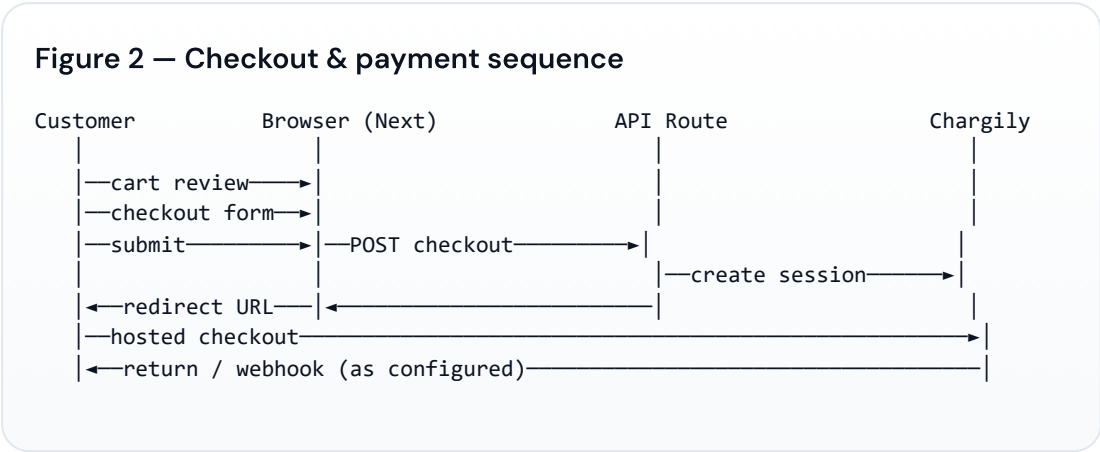
Better Auth is instantiated in `src/server/lib/auth.ts` with Drizzle adapter, custom session enrichment (e.g., phone, image lookups), Google provider, and trusted origins derived from environment. Client helpers live in `src/lib/auth-client.ts`.

Commerce, pricing modes, and inventory

Products carry list and optional promotional (*jomla*) prices with business rules documented in schema comments. Availability uses numeric stock representation suitable for extension to quantity-based fulfillment.

Checkout, logistics fields, and payments

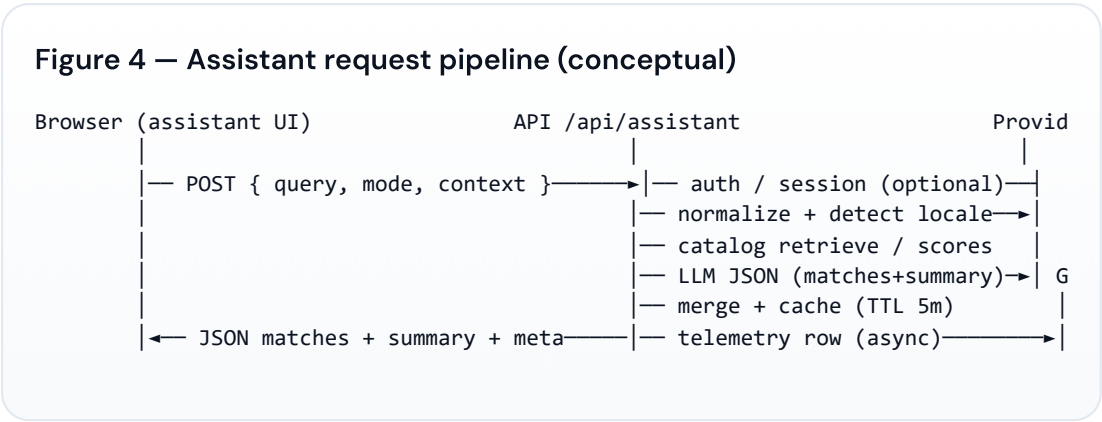
Checkout captures Algerian administrative geography (wilaya, commune) and optional customer notes. Payment initiation occurs server-side through Chargily’s REST API; the browser is redirected to hosted checkout, minimizing PCI scope on Souma infrastructure.



AI assistants: behavior and safeguards

The assistant route loads the in-repo catalog (shopData), normalizes queries (including transliteration), optionally calls Gemini 2.0 Flash and/or OpenRouter models, merges LLM scores with deterministic signals, caches hot responses for five minutes, and returns structured matches plus a natural-language summary. Missing API keys produce clear, localized guidance rather than opaque failures.

Telemetry captures request identifiers, provider metadata, cache status, matched identifiers, and click-through events for offline evaluation.



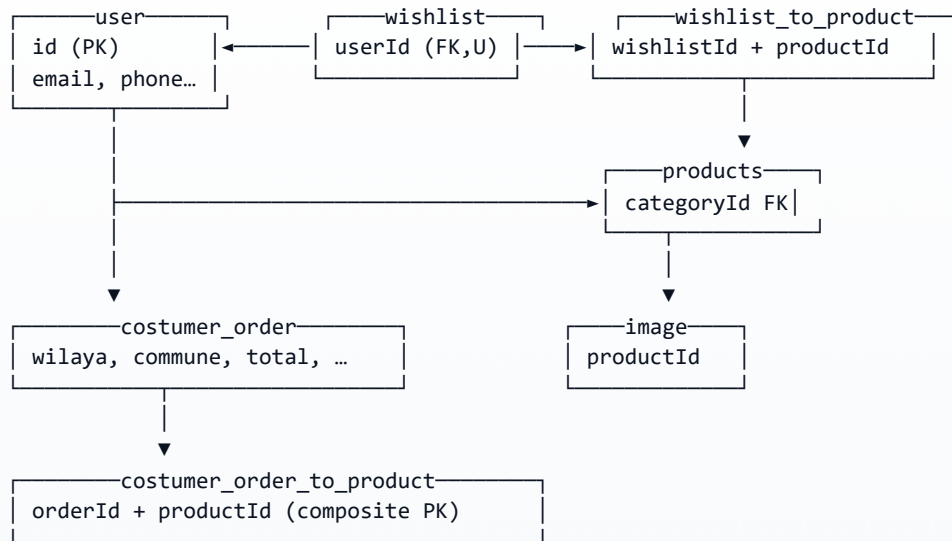
Analytics: shopping sequences

The `shopping_sequence` entity records session-keyed funnels with trigger labels, lifecycle status, and timestamps for product visits and endings. Admin APIs expose these rows for operational review.

Data architecture and persistence

PostgreSQL holds canonical commerce entities, Better Auth tables, reviews, feedback, sequences, and assistant telemetry. Drizzle migrations under `drizzle/` version additive changes.

Figure 3 — Core entity relationships (simplified)



API surface (summary)

Server routes are implemented as Next.js Route Handlers returning JSON or redirects. The appendix lists every discovered route file in this repository snapshot.

Security, privacy, and compliance posture

Secrets never ship to the client bundle. OAuth uses Google web client credentials. Payment secrets are server-only. Telemetry may contain search text—treat as sensitive operational data with retention controls.

Deployment model

Typical deployment targets Vercel or compatible platforms with environment variables mirroring `.env.example`. Database connectivity uses pooled Postgres URLs suitable for serverless drivers.

Testing strategy (recommendations)

Recommended layers: unit tests for pure utilities (price math, normalization), contract tests for Route Handlers with mocked Drizzle, Playwright smoke for checkout happy-path in staging, and periodic load tests on assistant endpoints.

Regression matrix (starter). Cart persistence across reload; assistant with keys absent/present; OAuth callback on localhost vs production origin; Chargily redirect URLs with and without trailing slashes; sequence lifecycle when user opens multiple tabs; wishlist FK cascade on user delete.

Test data hygiene. Use disposable Postgres schemas or docker-compose for CI, seed minimal categories/products, and rotate OAuth test users to avoid rate limits on Google's consent screen during automated runs.

Roadmap themes

1. Hardening auth gates on admin paths
2. Inventory reservation service
3. Richer CMS for blogs
4. Assistant evaluation dashboard
5. Webhook-driven order fulfillment status

Each theme should ship with measurable KPIs: e.g., assistant NDCG@k on held-out queries, checkout conversion rate by wilaya, median time-to-first-byte on PDP, and error budget for payment callbacks.

Internationalization, locales, and assistant copy decks

Souma's customer-facing UI is predominantly French/Arabic/Darija-friendly in merchandising copy, while the assistant route codifies **four response locales**—Modern Standard Arabic, French, Algerian Darija (Latin), and English—selected by script detection, provider-reported language, and regex heuristics tuned for Maghrebi French and Darija markers.

Template banks centralize operational messages (missing API keys, rate limits, empty queries, catalog-only fallback). This pattern keeps UX consistent even when the LLM narrative varies, and simplifies legal review of deterministic strings.

Future i18n work could externalize UI strings into message catalogs (e.g., next-intl) while preserving assistant locale detection as a separate concern from static chrome translation.

Repository layout and layering conventions

The repository follows a pragmatic vertical slice: `src/app` for routing and route handlers, `src/components` for UI modules grouped by domain (Shop, Checkout, Cart, Common), `src/server` for auth/db/data-access, `src/redux` for client state, and `src/lib` for cross-cutting utilities. This mirrors how teams onboard: start at a route, descend into components, drop to server helpers when persistence is involved.

Keeping Drizzle schema centralized avoids drift between “types” and “truth”. Migrations in `drizzle/*.sql` should be treated as immutable history once applied to production.

Catalog data model and shop presentation

Products are relational rows with slugs for clean URLs, joined categories, and optional promotional pricing. The storefront also consumes curated shopData for deterministic assistant retrieval—an intentional dual path: relational data for transactional integrity, curated JSON for fast LLM grounding without N+1 queries at inference time.

Merchandising components emphasize photography, manufacturer, and category breadcrumbs to reduce uncertainty before add-to-cart.

Product detail page: composition and assistants

The PDP composes gallery, price modes, description, reviews, and contextual assistant entry points. Product-scoped assistant payloads include title, availability hints, and category to steer retrieval toward compatible substitutes and accessories.

Separating global assistant chrome from PDP assistant reduces accidental context leakage and improves relevance metrics in telemetry.

Cart, persistence, and client state lifecycle

Redux holds authoritative client cart state; persistence middleware writes through to storage after meaningful transitions. Hydration strategies must avoid double-counting during SSR handoff—Souma’s approach favors client reconciliation with stable keys to prevent flicker when session state and anonymous carts meet at login.

Checkout UX: wilaya, commune, shipping presentation

Algerian wilaya and commune fields align with last-mile carrier expectations. The checkout surface presents **Yalidine Express** branding in the shipping method presentation layer, signaling fulfillment partner intent to shoppers even when rate tables are simplified in early versions.

Payments: Chargily integration principles

Server-created checkout sessions ensure secrets never leave trusted compute. Amounts and success/failure URLs should be derived from trusted cart recomputation on the server, not from client-submitted totals alone. Idempotency keys for order creation pair naturally with payment intent identifiers.

Wishlist domain model and UX

Wishlists are modeled as one row per user with a join table for products, enabling efficient uniqueness constraints and cascade deletes. UX surfaces highlight saved items and nudge conversion through return visits.

Reviews, ratings, and moderation considerations

Reviews bind to users and carry integer ratings and free-text comments. Moderation policies (profanity filters, report button, admin quarantine) are organizational choices; technically the schema supports append-only consumer history suitable for audit.

Site feedback instrument

Holistic site feedback complements product reviews by capturing satisfaction with navigation, speed, and support. Aggregating feedback alongside assistant telemetry can explain macro conversion changes.

Blog subsystem and editorial workflow

Blog routes provide editorial grid/detail layouts with optional sidebars for long-form storytelling—useful for launches, lookbooks, and merchant education. A headless CMS integration would be a natural evolution without changing public URLs.

Legal, trust, and policy surfaces

Terms, privacy, and refund policies establish baseline consumer trust. These pages should be versioned in git with change logs when material terms shift (returns window, data retention, payment dispute flows).

Performance engineering checklist

- Image formats and sizes for hero and PDP galleries
- Defer non-critical assistant bundles until interaction
- Memoize expensive selectors; avoid rerendering entire grids on cart tick
- Prefer server components for static policy pages
- Monitor API cold starts on Vercel for assistant route

Accessibility baseline

Semantic landmarks, focus management for modals (quick view, assistant), sufficient color contrast on navy backgrounds, and keyboard operability for primary CTAs are baseline expectations. Automated checks (axe) plus manual screen reader passes on checkout are recommended before major releases.

Engineering process: migrations, typing, and CI

Adopt a migration-first workflow: local `drizzle-kit generate` after schema edits, peer review SQL, apply in staging, snapshot restore drills before production.

TypeScript strictness should ratchet over time; new modules should avoid any in auth and payment boundaries.

Glossary and extended definitions

App Router

Next.js file-system routing with layouts and nested loading boundaries.

Better Auth

Library handling sessions, OAuth, and credential storage with Drizzle adapter.

Chargily

Regional payment gateway integration used for hosted checkout.

Drizzle

Type-safe SQL builder and migration toolkit for Postgres.

jomla

Promotional price mode per Souma business rules.

Route Handler

`route.ts` file exporting HTTP method functions in the App Router.

shopping_sequence

Funnel analytics entity keyed by session with lifecycle statuses.

telemetry

Assistant search and click events stored for offline evaluation.

Appendices

Detailed tables and schema excerpts follow. These sections intentionally mirror repository structure to keep the report auditable against git history.

Appendix A — Primary page routes

Route	Description
/	Marketing home, hero, categories, featured products
/signin	Better Auth sign-in (Google + credentials flows)
/signup	Account registration
/forgot-password	Password recovery entry
/shop-with-sidebar	Catalog grid with sidebar filters
/shop-without-sidebar	Catalog grid without sidebar
/shop-details	Product detail (query-driven)
/category/[slug]	Category landing with product listing
/cart	Cart review, line items, proceed to checkout
/checkout	Wilaya/commune, notes, Chargily payment handoff
/wishlist	Saved products (authenticated)
/my-account	Profile and account management
/recently-viewed	Session/history-based product recall
/contact	Contact form / store information
/faq	Frequently asked questions
/terms-of-use	Legal terms
/privacy-policy	Privacy policy
/refund-policy	Refund policy
/mail-success	Post-transaction confirmation
/error	Error boundary page
/sequence	Shopping sequence analytics (admin-style view)
/admin	Administrative tools (sequences, etc.)
/blogs/blog-grid	Blog index grid
/blogs/blog-grid-with-sidebar	Blog grid with sidebar

Route	Description
/blogs/blog-details	Single blog article
/blogs/blog-details-with-sidebar	Blog article with sidebar

Appendix B — API route handlers (inventory)

Path	Description
/api/auth/[...all]	Better Auth catch-all: session, OAuth callbacks
/api/assistant	POST: multilingual smart assistant (Gemini / OpenRouter), catalog + LLM ranking
/api/assistant/telemetry	Assistant search / click telemetry ingestion
/api/payments/chargily/checkout	Creates Chargily checkout session, redirects
/api/reviews	Product reviews CRUD / listing
/api/feedbacks	Site-wide feedback capture
/api/sequence/start	Starts shopping_sequence funnel row
/api/sequence/visit-product	Marks product visit within sequence
/api/sequence/end	Ends sequence (purchase or leave)
/api/admin/sequences	Admin listing / inspection of sequences

Appendix C — Use case catalog

ID	Name	Primary actor	Brief description
UC-01	Visitor browses home	Guest	Views hero, categories, and featured products without signing in.
UC-02	Visitor opens shop grid	Guest	Navigates to shop-with-sidebar or shop-without-sidebar and scrolls catalog.
UC-03	Filter products by category	Guest	Uses category chips or category pages to narrow catalog.
UC-04	Open product detail	Guest	Clicks product card; loads shop-details with images, price modes, description.
UC-05	Toggle price mode (detail vs jomla)	Guest/User	Global PriceModeProvider switches how list prices are interpreted (reference vs promo).
UC-06	Add to cart	Guest/User	Dispatches Redux cart slice; toast feedback; cart badge updates.
UC-07	Adjust quantity in cart	Guest/User	Updates line quantities with validation against stock fields.
UC-08	Remove from cart	Guest/User	Removes line item; persists via CartPersistence.
UC-09	Persist cart across reload	Guest/User	CartPersistence hydrates Redux from storage without flicker.
UC-10	Open quick view modal	Guest/User	QuickView slice drives modal with product summary.
UC-11	Add to wishlist	User	Requires session; wishlist tables in Postgres.
UC-12	View wishlist	User	Dedicated wishlist page lists saved SKUs.
UC-13	Sign in with Google	Guest	Better Auth + Google OAuth; profile hydrated.
UC-14	Sign in with email/password	Guest	Credential provider via Better Auth.
UC-15	Sign up	Guest	Creates user row, verification flags per configuration.

ID	Name	Primary actor	Brief description
UC-16	Forgot password	Guest	Initiates recovery flow (provider-dependent).
UC-17	View my account	User	Profile, orders history hooks (as implemented).
UC-18	Start checkout	User/Guest	Checkout form: wilaya, commune, optional note.
UC-19	Pay with Chargily	User/Guest	Server creates Chargily session; browser redirects to hosted checkout.
UC-20	Return from payment	User/Guest	Redirect URLs use NEXT_PUBLIC_APP_URL; order persisted.
UC-21	Submit product review	User	Reviews API + product_review table.
UC-22	Read reviews on product	Guest/User	Rendered list with ratings aggregate.
UC-23	Submit site feedback	User	site_feedback table via API.
UC-24	Contact store	Guest/User	Contact page messaging (implementation-specific).
UC-25	Read policies	Guest/User	Static legal pages: terms, privacy, refund.
UC-26	Browse blog	Guest/User	Marketing / content pages under /blogs/*.
UC-27	FAQ self-service	Guest/User	FAQ page reduces support load.
UC-28	Recently viewed products	Guest/User	recently-viewed page surfaces local/session history.
UC-29	Smart shopping assistant (global)	Guest/User	Floating assistant; POST /api/assistant; multilingual replies.
UC-30	Product page assistant	Guest/User	Contextual assistant with on-page product metadata.
UC-31	Assistant telemetry — search	System	Logs normalized query, provider, model, cache, counts.

ID	Name	Primary actor	Brief description
UC-32	Assistant telemetry — click	User	Logs clicked recommendation position for tuning.
UC-33	Sequence start	System	sequence/start records funnel with trigger metadata.
UC-34	Sequence product visit	System	visit-product stamps product_visited_at.
UC-35	Sequence end purchase	System	end marks purchase outcome.
UC-36	Sequence end abandon	System	end marks left / superseded states.
UC-37	Admin inspect sequences	Admin	admin/sequences API + UI page.
UC-38	Middleware session cookie check	System	middleware.ts optimistic redirect for /dashboard matcher.
UC-39	Session provider (client)	Guest/User	React session context from Better Auth client.
UC-40	Image gallery on product	Guest/User	Additional images from image table / UI carousel.
UC-41	Stock / availability messaging	Guest/User	instock field drives availability copy.
UC-42	Manufacturer & category display	Guest/User	Schema fields rendered in PDP.
UC-43	Search normalization (assistant)	System	Transliteration + language heuristics before retrieval.
UC-44	LLM provider fallback chain	System	Gemini primary; OpenRouter configurable; graceful degradation.
UC-45	Rate limit user messaging	Guest/User	Localized strings when providers throttled.
UC-46	Mail success confirmation	User	Post-checkout positive acknowledgment page.
UC-47	Error page navigation	Guest/User	Centralized error UX.

ID	Name	Primary actor	Brief description
UC-48	SEO / metadata per layout	Guest/User	Next.js metadata API on routes (as configured).
UC-49	Redux DevTools (dev)	Developer	RTK store exposes standard tooling in development.
UC-50	Production deploy on Vercel	Operator	Serverless functions for API routes; env secrets in project settings.
UC-51	Theme toggle (light/dark)	Guest/User	next-themes integration where wired in layout.
UC-52	Swiper-based merchandising carousels	Guest/User	Marketing sliders for collections and promos.
UC-53	Category slug resolution	Guest/User	Dynamic segment category/[slug] hydrates listing.
UC-54	Order line composition	System	costumer_order_to_product rows mirror cart at checkout.
UC-55	Product images secondary gallery	Guest/User	imageTable joins power additional PDP media.
UC-56	Phone uniqueness enforcement	System	user.phone unique constraint supports support desk lookups.
UC-57	Role field on user	System	Default user role; extensible for admin elevation.
UC-58	Session expiry handling	User	Better Auth session rows with expires_at semantics.
UC-59	OAuth account linkage	User	account table stores provider tokens per Better Auth model.
UC-60	Email verification flags	User	email_verified boolean on user as per adapter mapping.
UC-61	Verification tokens	System	verification table for magic links / OTP flows.
UC-62	Assistant detail vs jomla mode	Guest/User	mode flag steers catalog matching and pricing context.
UC-63	Assistant empty-query guard	Guest/User	Localized prompt when query blank.

ID	Name	Primary actor	Brief description
UC-64	Assistant catalog fallback	System	Deterministic retrieval when LLM unavailable.
UC-65	Assistant response cache	System	In-memory TTL map keyed by normalized query.
UC-66	Sequence superseded state	System	New funnel run marks prior rows superseded.
UC-67	Sequence trigger taxonomy	Operator	trigger_type + trigger_label explain entry points.
UC-68	Admin sequences JSON export mindset	Admin	Rows consumable by BI pipelines (future).
UC-69	Checkout shipping method display	Guest/User	Yalidine Express branding in ShippingMethod UI.
UC-70	Hydration-safe cart badge	Guest/User	Client-only counts after persistence hydration.
UC-71	Product local id in reviews	System	Legacy/local id field aligns reviews with catalog rows.
UC-72	Strikethrough reference pricing	Guest/User	When jomlaPrice set, UI shows reference + promo story.
UC-73	Manufacturer trust signal	Guest/User	manufacturer field displayed on PDP.
UC-74	Low-stock edge messaging	Guest/User	instock thresholds can inform urgency copy.
UC-75	404 / error recovery	Guest/User	Dedicated error page route.
UC-76	Mail success analytics hook	Operator	Confirmation page can attach conversion pixels.
UC-77	Contact lead capture	Guest/User	Inbound inquiries for B2B or support.
UC-78	FAQ reduces tickets	Guest/User	Self-serve answers before human support.
UC-79	Blog SEO landing	Guest/User	Long-form content attracts organic traffic.

ID	Name	Primary actor	Brief description
UC-80	Policy audit trail	Compliance	Static pages demonstrate consumer disclosures.

Appendix D — Risk register (excerpt)

ID	Area	Description	Severity	Mitigation
R-01	Payment integration	Third-party Chargily availability and webhook timing must be monitored.	High	Retries, idempotent order writes, alerting
R-02	LLM cost & abuse	Assistant endpoints can incur token cost; prompt injection surface.	Medium	Rate limits, max tokens, query sanitization
R-03	Auth middleware comment	Code notes cookie check is optimistic; routes must re-validate.	Medium	Central requireAuth helper for sensitive APIs
R-04	PII in telemetry	Assistant telemetry stores queries; retention policy needed.	Medium	TTL cleanup, anonymization
R-05	Stock oversell	Race between cart and checkout vs inventory.	Medium	Transactional stock decrement
R-06	Env misconfiguration	Missing GOOGLE_API_KEY / OPENROUTER degrades assistant.	Low	Health check + UI copy
R-07	Database migrations	Drift between drizzle and production schema.	Medium	CI migration gate
R-08	Third-party LLM availability	Provider outages affect assistant quality.	Medium	Multi-provider fallback, cached fallbacks
R-09	Client bundle size	Heavy client islands can regress LCP.	Low	Code-split assistants, lazy modals

Appendix E — Environment template (.env.example)

```
DATABASE_URL='postgres://role:password@localhost:5432/db-name'
BETTER_AUTH_SECRET="Random value used by the library for encryption and
generating hashes. you can use something like openssl"
BETTER_AUTH_URL=http://localhost:3000 #Base URL of your app
GOOGLE_CLIENT_ID="" # Google OAuth Web Client ID
GOOGLE_CLIENT_SECRET="" # Google OAuth Web Client Secret
CHARGILY_SECRET_KEY="" # Chargily secret key (test or live)
CHARGILY_API_BASE_URL="https://pay.chargily.net/test/api/v2" # Test mode base
URL
NEXT_PUBLIC_APP_URL="http://localhost:3000" # Public app URL for payment
redirects
```

Appendix F — Drizzle schema excerpts (generated)

Column bodies are copied from `src/server/db/schema.ts` for audit traceability.
Composite keys and references appear as declared in source.

Table `user` (`user`)

```
id: text("id").primaryKey(),
name: text("name").notNull(),
lastname: text("last_name").notNull(),
phone: varchar("phone", { length: 10 }).notNull().unique(),
role: text("role").notNull().default("user"),
email: text("email").notNull().unique(),
emailVerified: boolean("email_verified")
  .$defaultFn(() => false)
  .notNull(),
image: text("image"),
createdAt: timestamp("created_at")
  .$defaultFn(() => /* @@PURE__ */ new Date())
  .notNull(),
updatedAt: timestamp("updated_at")
  .$defaultFn(() => /* @@PURE__ */ new Date())
  .notNull(),
}
```

Table `session` (`session`)

```
id: text("id").primaryKey(),
expiresAt: timestamp("expires_at").notNull(),
token: text("token").notNull().unique(),
createdAt: timestamp("created_at").notNull(),
updatedAt: timestamp("updated_at").notNull(),
ipAddress: text("ip_address"),
userAgent: text("user_agent"),
userId: text("user_id")
  .notNull()
  .references(() => user.id, { onDelete: "cascade" }),
}
```

Table `account` (`account`)

```
id: text("id").primaryKey(),
accountId: text("account_id").notNull(),
providerId: text("provider_id").notNull(),
userId: text("user_id")
  .notNull()
  .references(() => user.id, { onDelete: "cascade" }),
accessToken: text("access_token"),
refreshToken: text("refresh_token"),
```



```

idToken: text("id_token"),
accessTokenExpiresAt: timestamp("access_token_expires_at"),
refreshTokenExpiresAt: timestamp("refresh_token_expires_at"),
scope: text("scope"),
password: text("password"),
createdAt: timestamp("created_at").notNull(),
updatedAt: timestamp("updated_at").notNull(),
}

```

Table verification (verification)

```

id: text("id").primaryKey(),
identifier: text("identifier").notNull(),
value: text("value").notNull(),
expiresAt: timestamp("expires_at").notNull(),
createdAt: timestamp("created_at").$defaultFn(
  () => /* @__PURE__ */ new Date()
),
updatedAt: timestamp("updated_at").$defaultFn(
  () => /* @__PURE__ */ new Date()
),
}

```

Table wishlist (wishlistTable)

```

id: uuid().primaryKey().defaultRandom(),
// one wishlist has one user
userId: text()
  .references(() => user.id, { onDelete: "cascade" })
  .unique(),
}

```

Table products (productsTable)

```

id: uuid().primaryKey().defaultRandom(),
slug: varchar({ length: 255 }).notNull().unique(),
title: varchar({ length: 255 }).notNull(),
mainimage: varchar({ length: 255 }).notNull(),
/** Shown as strikethrough "standard" when jomlaPrice is set; otherwise the
sole list price */
price: integer().notNull().default(0),
/** Souma (promo) price – when set, customer pays this and price is +20%
reference */
jomlaPrice: integer("jomla_price"),
rating: integer().notNull().default(0),
description: varchar({ length: 255 }).notNull(),
manufacturer: varchar({ length: 255 }).notNull(),
// in stock attribute for normal ecommerce but for the project with w4t3r
// replace it with quantity till the product is ordered from the store
instock: integer().notNull().default(1),
categoryId: uuid()
  .references(() => categoryTable.id, { onDelete: "cascade" })

```

```
.NotNull(),  
}
```

Table wishlist_to_product (wishlist_to_productTable)

```
wishlistId: uuid()  
  .references(() => wishlistTable.id)  
  .NotNull(),  
productId: uuid()  
  .references(() => productsTable.id)  
  .NotNull(),  
addedAt: timestamp({ mode: "date" }).NotNull().defaultNow(),  
}
```

Table image (imageTable)

```
id: uuid().primaryKey().defaultRandom(),  
productId: uuid()  
  .references(() => productsTable.id)  
  .NotNull(),  
image: varchar({ length: 255 }).NotNull(),  
}
```

Table category (categoryTable)

```
id: uuid().primaryKey().defaultRandom(),  
name: varchar({ length: 255 }).NotNull(),  
}
```

Table shopping_sequence (shoppingSequenceTable)

```
id: uuid().primaryKey().defaultRandom(),  
sessionKey: varchar("session_key", { length: 64 }).NotNull(),  
userId: text("user_id").references(() => user.id, { onDelete: "set null" }),  
triggerType: varchar("trigger_type", { length: 32 }).NotNull(),  
triggerLabel: text("trigger_label").NotNull(),  
/** active | superseded | ended_left | ended_purchase */  
status: varchar("status", { length: 32 }).NotNull().default("active"),  
productVisitedAt: timestamp("product_visited_at", { mode: "date" }),  
startedAt: timestamp("started_at", { mode: "date" }).NotNull().defaultNow(),  
endedAt: timestamp("ended_at", { mode: "date" }),  
}
```

Table costumer_order (costumer_orderTable)

```
id: uuid().primaryKey().defaultRandom(),  
userId: text()
```

```

        .references(() => user.id)
        .notNull(),
    wilaya: varchar({ length: 255 }).notNull(),
    commune: varchar({ length: 255 }).notNull(),
    note: varchar({ length: 255 }),
    datetime: timestamp({ mode: "date" }).notNull().defaultNow(),
    total: integer().notNull().default(0),
}

```

Table **costumer_order_to_product** (costumer_order_to_productTable)

```

orderId: uuid()
    .references(() => costumer_orderTable.id)
    .notNull(),
productId: uuid()
    .references(() => productsTable.id)
    .notNull(),
quantity: integer().notNull().default(1),
}

```

Table **product_review** (productReviewTable)

```

id: text("id").primaryKey(),
productLocalId: integer("product_local_id").notNull(),
productTitle: text("product_title").notNull(),
userId: text("user_id")
    .notNull()
    .references(() => user.id, { onDelete: "cascade" }),
rating: integer("rating").notNull(),
comment: text("comment").notNull(),
createdAt: timestamp("created_at", { mode: "date" }).notNull().defaultNow(),
}

```

Table **site_feedback** (siteFeedbackTable)

```

id: text("id").primaryKey(),
userId: text("user_id")
    .notNull()
    .references(() => user.id, { onDelete: "cascade" }),
rating: integer("rating").notNull(),
comment: text("comment").notNull(),
createdAt: timestamp("created_at", { mode: "date" }).notNull().defaultNow(),
}

```

Table **assistant_search_telemetry** (assistantSearchTelemetryTable)

```

id: uuid().primaryKey().defaultRandom(),
eventType: varchar("event_type", { length: 32 }).notNull(),
requestId: varchar("request_id", { length: 64 }).notNull(),

```

```
sessionKey: varchar("session_key", { length: 64 }),
userId: text("user_id").references(() => user.id, { onDelete: "set null" }),
mode: varchar("mode", { length: 16 }).notNull().default("detail"),
rawQuery: text("raw_query"),
normalizedQuery: text("normalized_query"),
detectedLanguage: varchar("detected_language", { length: 64 }),
provider: varchar("provider", { length: 64 }),
model: varchar("model", { length: 128 }),
error: text("error"),
cacheStatus: varchar("cache_status", { length: 16 }),
resultCount: integer("result_count").notNull().default(0),
matchedIdsJson: text("matched_ids_json"),
clickedProductId: integer("clicked_product_id"),
clickedPosition: integer("clicked_position"),
createdAt: timestamp("created_at", { mode: "date" }).notNull().defaultNow(),
}
```

Appendix G — Source file index (generated from disk)

Path (under repo)	Classification
src/app/(site)/(pages)/admin/AdminPanels.tsx	TypeScript module
src/app/(site)/(pages)/admin/EditProductModal.tsx	TypeScript module
src/app/(site)/(pages)/admin/actions.ts	TypeScript module
src/app/(site)/(pages)/admin/page.tsx	App Router page
src/app/(site)/(pages)/admin/product-form-ui.tsx	TypeScript module
src/app/(site)/(pages)/cart/page.tsx	App Router page
src/app/(site)/(pages)/category/[slug]/page.tsx	App Router page
src/app/(site)/(pages)/checkout/page.tsx	App Router page
src/app/(site)/(pages)/contact/page.tsx	App Router page
src/app/(site)/(pages)/error/page.tsx	App Router page
src/app/(site)/(pages)/faq/page.tsx	App Router page
src/app/(site)/(pages)/forgot-password/page.tsx	App Router page
src/app/(site)/(pages)/mail-success/page.tsx	App Router page
src/app/(site)/(pages)/my-account/page.tsx	App Router page
src/app/(site)/(pages)/privacy-policy/page.tsx	App Router page
src/app/(site)/(pages)/recently-viewed/page.tsx	App Router page
src/app/(site)/(pages)/refund-policy/page.tsx	App Router page
src/app/(site)/(pages)/sequence/page.tsx	App Router page
src/app/(site)/(pages)/shop-details/page.tsx	App Router page
src/app/(site)/(pages)/shop-with-sidebar/page.tsx	App Router page
src/app/(site)/(pages)/shop-without-sidebar/page.tsx	App Router page
src/app/(site)/(pages)/signin/action.ts	TypeScript module
src/app/(site)/(pages)/signin/page.tsx	App Router page
src/app/(site)/(pages)/signin/signin-form.tsx	TypeScript module
src/app/(site)/(pages)/signup/action.ts	TypeScript module
src/app/(site)/(pages)/signup/page.tsx	App Router page
src/app/(site)/(pages)/signup/signup-form.tsx	TypeScript module
src/app/(site)/(pages)/terms-of-use/page.tsx	App Router page
src/app/(site)/(pages)/wishlist/page.tsx	App Router page
src/app/(site)/blogs/blog-details-with-sidebar/page.tsx	App Router page
src/app/(site)/blogs/blog-details/page.tsx	App Router page
src/app/(site)/blogs/blog-grid-with-sidebar/page.tsx	App Router page
src/app/(site)/blogs/blog-grid/page.tsx	App Router page

Path (under repo)	Classification
src/app/(site)/layout.tsx	App Router layout
src/app/(site)/page.tsx	App Router page
src/app/api/admin/sequences/route.ts	Route Handler (API)
src/app/api/assistant/route.ts	Route Handler (API)
src/app/api/assistant/telemetry/route.ts	Route Handler (API)
src/app/api/auth/[...all]/route.ts	Route Handler (API)
src/app/api/feedbacks/route.ts	Route Handler (API)
src/app/api/payments/chargily/checkout/route.ts	Route Handler (API)
src/app/api/reviews/route.ts	Route Handler (API)
src/app/api/sequence/_cookie.ts	Route Handler (API)
src/app/api/sequence/end/route.ts	Route Handler (API)
src/app/api/sequence/start/route.ts	Route Handler (API)
src/app/api/sequence/visit-product/route.ts	Route Handler (API)
src/app/context/CartSidebarModalContext.tsx	TypeScript module
src/app/context/PreviewSliderContext.tsx	TypeScript module
src/app/context/PriceModeContext.tsx	TypeScript module
src/app/context/QuickViewModalContext.tsx	TypeScript module
src/app/context/SessionProvider.tsx	TypeScript module
src/components/Admin/SequencesLiveTable.tsx	React component / UI
src/components/Auth/Signin/index.tsx	React component / UI
src/components/Auth/Signup/index.tsx	React component / UI
src/components/Blog/BlogItem.tsx	React component / UI
src/components/Blog/Categories.tsx	React component / UI
src/components/Blog/LatestPosts.tsx	React component / UI
src/components/Blog/LatestProducts.tsx	React component / UI
src/components/Blog/SearchForm.tsx	React component / UI
src/components/BlogDetails/index.tsx	React component / UI
src/components/BlogDetailsWithSidebar/index.tsx	React component / UI
src/components/BlogGrid/blogData.ts	React component / UI
src/components/BlogGrid/index.tsx	React component / UI
src/components/BlogGridWithSidebar/index.tsx	React component / UI
src/components/Cart/Discount.tsx	React component / UI
src/components/Cart/OrderSummary.tsx	React component / UI
src/components/Cart/SingleItem.tsx	React component / UI
src/components/Cart/index.tsx	React component / UI

Path (under repo)	Classification
src/components/Category/index.tsx	React component / UI
src/components/Checkout/Billing.tsx	React component / UI
src/components/Checkout/Coupon.tsx	React component / UI
src/components/Checkout/Login.tsx	React component / UI
src/components/Checkout/Notes.tsx	React component / UI
src/components/Checkout/OrderList.tsx	React component / UI
src/components/Checkout/PaymentMethod.tsx	React component / UI
src/components/Checkout/Shipping.tsx	React component / UI
src/components/Checkout/ShippingMethod.tsx	React component / UI
src/components/Checkout/index.tsx	React component / UI
src/components/Common/Breadcrumb.tsx	React component / UI
src/components/Common/CartPersistence.tsx	React component / UI
src/components/Common/CartSidebarModal/EmptyCart.tsx	React component / UI
src/components/Common/CartSidebarModal/SingleItem.tsx	React component / UI
src/components/Common/CartSidebarModal/index.tsx	React component / UI
src/components/Common/Newsletter.tsx	React component / UI
src/components/Common/PreLoader.tsx	React component / UI
src/components/Common/PreviewSlider.tsx	React component / UI
src/components/Common/ProductItem.tsx	React component / UI
src/components/Common/QuickViewModal.tsx	React component / UI
src/components/Common/ScrollToTop.tsx	React component / UI
src/components/Common/SequenceRouteWatcher.tsx	React component / UI
src/components/Common/SmartShoppingAssistant.tsx	React component / UI
src/components/Common/Spinner.tsx	React component / UI
src/components/Contact/index.tsx	React component / UI
src/components/Error/index.tsx	React component / UI
src/components/Footer/index.tsx	React component / UI
src/components/Header/CustomSelect.tsx	React component / UI
src/components/Header/Dropdown.tsx	React component / UI
src/components/Header/index.tsx	React component / UI
src/components/Header/menuData.ts	React component / UI
src/components/Home/BestSeller/SingleItem.tsx	React component / UI
src/components/Home/BestSeller/index.tsx	React component / UI
src/components/Home/Categories/SingleItem.tsx	React component / UI
src/components/Home/Categories/categoryData.ts	React component / UI

Path (under repo)	Classification
src/components/Home/Categories/index.tsx	React component / UI
src/components/Home/Countdown/index.tsx	React component / UI
src/components/Home/Hero/HeroCarousel.tsx	React component / UI
src/components/Home/Hero/HeroFeature.tsx	React component / UI
src/components/Home/Hero/index.tsx	React component / UI
src/components/Home/NewArrivals/index.tsx	React component / UI
src/components/Home/PromoBanner/index.tsx	React component / UI
src/components/Home/Testimonials/SingleItem.tsx	React component / UI
src/components/Home/Testimonials/index.tsx	React component / UI
src/components/Home/Testimonials/testimonialsData.ts	React component / UI
src/components/Home/index.tsx	React component / UI
src/components/MailSuccess/index.tsx	React component / UI
src/components/MyAccount/AddressModal.tsx	React component / UI
src/components/MyAccount/action.ts	React component / UI
src/components/MyAccount/index.tsx	React component / UI
src/components/MyAccount/tabsData.tsx	React component / UI
src/components/Orders/EditOrder.tsx	React component / UI
src/components/Orders/OrderActions.tsx	React component / UI
src/components/Orders/OrderDetails.tsx	React component / UI
src/components/Orders/OrderModal.tsx	React component / UI
src/components/Orders/SingleOrder.tsx	React component / UI
src/components/Orders/index.tsx	React component / UI
src/components/Orders/ordersData.tsx	React component / UI
src/components/Shop/SingleGridItem.tsx	React component / UI
src/components/Shop/SingleListItem.tsx	React component / UI
src/components/Shop/shopData.ts	React component / UI
src/components/ShopDetails/ProductPageAssistant.tsx	React component / UI
src/components/ShopDetails/RecentlyViewd/index.tsx	React component / UI
src/components/ShopDetails/ReviewsTab.tsx	React component / UI
src/components/ShopDetails/index.tsx	React component / UI
src/components/ShopWithSidebar/CategoryDropdown.tsx	React component / UI
src/components/ShopWithSidebar/ColorsDropdwon.tsx	React component / UI
src/components/ShopWithSidebar/CustomSelect.tsx	React component / UI
src/components/ShopWithSidebar/GenderDropdown.tsx	React component / UI
src/components/ShopWithSidebar/PriceDropdown.tsx	React component / UI

Path (under repo)	Classification
src/components/ShopWithSidebar/Progress.tsx	React component / UI
src/components/ShopWithSidebar/SizeDropdown.tsx	React component / UI
src/components/ShopWithSidebar/index.tsx	React component / UI
src/components/ShopWithoutSidebar/index.tsx	React component / UI
src/components/Wishlist/SingleItem.tsx	React component / UI
src/components/Wishlist/index.tsx	React component / UI
src/components/ui/form.tsx	React component / UI
src/components/ui/input.tsx	React component / UI
src/components/ui/label.tsx	React component / UI
src/components/ui/skeleton.tsx	React component / UI
src/components/ui/sonner.tsx	React component / UI
src/lib/auth-client.ts	Client or shared library
src/lib/price-mode.ts	Client or shared library
src/lib/product-content.ts	Client or shared library
src/lib/sequence-client.ts	Client or shared library
src/lib/utils.ts	Client or shared library
src/redux/features/cart-slice.ts	Redux slice or store
src/redux/features/product-details.ts	Redux slice or store
src/redux/features/quickView-slice.ts	Redux slice or store
src/redux/features/wishlist-slice.ts	Redux slice or store
src/redux/provider.tsx	Redux slice or store
src/redux/store.ts	Redux slice or store
src/server/assistant/telemetry-db.ts	Server, auth, or data access
src/server/data-access/product-catalog.ts	Server, auth, or data access
src/server/data-access/user.ts	Server, auth, or data access
src/server/db/index.ts	Server, auth, or data access
src/server/db/schema.ts	Server, auth, or data access
src/server/lib/action-helpers.ts	Server, auth, or data access
src/server/lib/auth.ts	Server, auth, or data access
src/server/reviews/reviews-db.ts	Server, auth, or data access
src/server/sequence/sequence-db.ts	Server, auth, or data access
src/types/Menu.ts	Type definitions
src/types/auth.ts	Type definitions
src/types/blogItem.ts	Type definitions
src/types/category.ts	Type definitions

Path (under repo)	Classification
src/types/product.ts	Type definitions
src/types/shopping-sequence.ts	Type definitions
src/types/testimonial.ts	Type definitions
src/types/user.ts	Type definitions
src/use-cases/user.ts	Application use case

Colophon. Typography pairs DM Sans (headings, UI) with Source Serif 4 (body).
Diagrams are ASCII for universal PDF fidelity. To regenerate: `node scripts/build-souma-application-report.mjs` then `node scripts/build-souma-application-report.mjs --pdf` with Playwright installed.